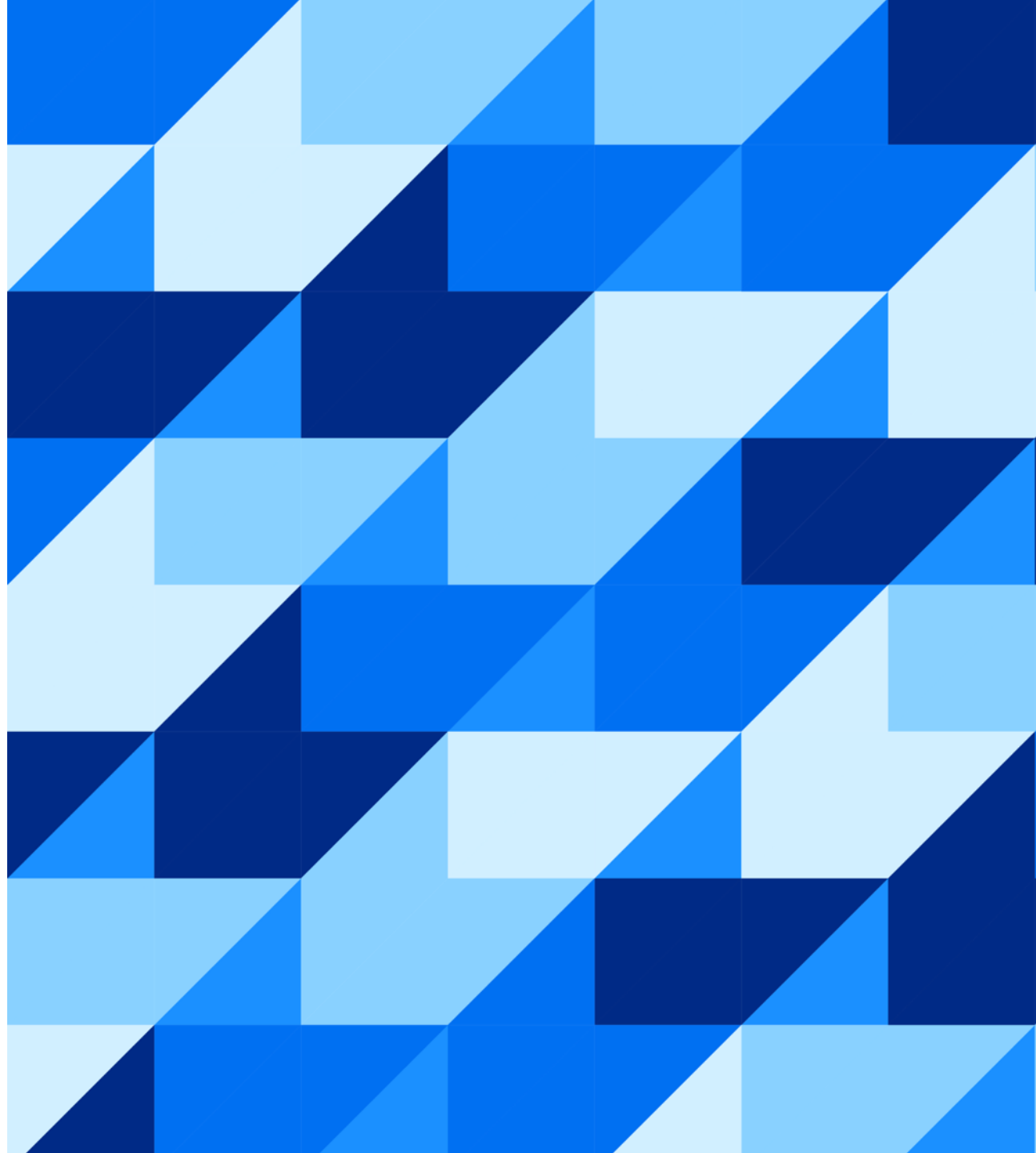




Joule's ABAP developer capabilities in ABAP Developer Tools

Sevdiye Linnhoff, Leon Knorr & Manuel Berning, SAP
June 05, 2025

Public



Agenda

1. Code Explain

- Use case introduction & demo
- Behind the scenes
- Measuring quality of ABAP Code Explanations

2. Code Prediction

- Use case introduction & demo
- Training an ABAP code suggestion model at SAP
- Measuring quality of ABAP code suggestions

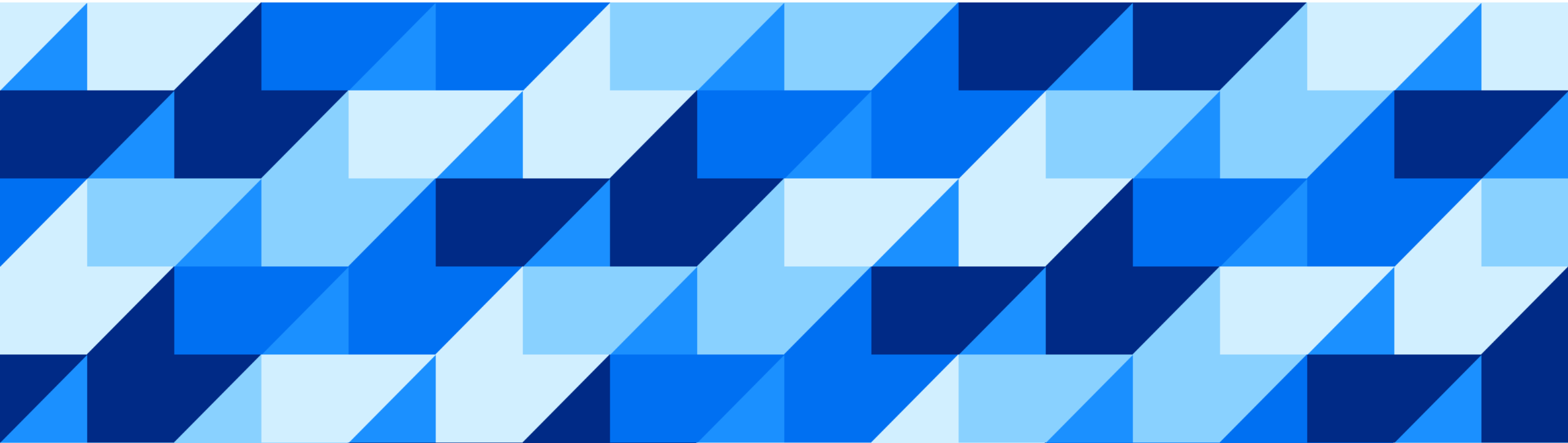
3. Context injection

- Challenges
- Identifying suitable context
- Architecture Overview

Code Explain

Sevdiye Linnhoff, SAP

June 05, 2025



Code Explain

Try it out now!

CL_MM_PUR_COMMON_PR_UTIL

1=class CL_MM_PUR_COMMON_PR_UTIL definition

2 public

3 final

4 create private .

5

6 public section.

7

8 interfaces IF_MM_PUR_COMMON_PR_UTIL .

9

10 class-methods GET_INSTANCE

11 returning

12 value(RO_INSTANCE) type ref to IF_MM_PUR_COMMON_PR_UTIL .

13 PROTECTED SECTION.

14 private section.

15

16 types:

17 BEGIN OF ty_s_pr_currency_buffer,

18 curr_config TYPE mmpur_req_d_pr_hdr_curr_src,

19 plant TYPE werks,

20 companycode TYPE bukrs,

21 purreqncurrency TYPE waers,

22 source_of_currency_text TYPE ddtex,

23 END OF ty_s_pr_currency_buffer .

24 types:

25 ty_t_pr_currency_buffer TYPE TABLE OF ty_s_pr_currency_buffer .

26 types:

27 BEGIN OF ty_s_curr_src_text,

28 curr_config type MMPUR_REQ_D_PR_HDR_CURR_SRC,

29 source_of_currency_text TYPE ddtex,

30 END OF ty_s_curr_src_text .

31 types:

32 ty_t_curr_src_text TYPE TABLE OF ty_s_curr_src_text .

33 types:

34 BEGIN OF ty_s_hierarchy_items.

35 INCLUDE TYPE r_purchasereqnitmhiernode.

36 TYPES purreqndraftuuid TYPE /bobf/uuid.

37 TYPES END OF ty_s_hierarchy_items .

38 types:

39 ty_t_hierarchy_items TYPE STANDARD TABLE OF ty_s_hierarchy_items WITH EMPTY KEY .

40

41 class-data MO_INSTANCE type ref to CL_MM_PUR_COMMON_PR_UTIL .

42 class-data MV_HDR_CURRENCY_CONFIG type MMPUR_REQ_D_PR_HDR_CURR_SRC .

43 class-data MT_PR_CURRENCY_BUFFER type TY_T_PR_CURRENCY_BUFFER .

44 constants C_USER_FIRST_ITEM_CURRENCY type MMPUR_REQ_D_PR_HDR_CURR_SRC value '4' ##NC

45 constants C_PR_ITEM_COMPANYCODE_CURR type MMPUR_REQ_D_PR_HDR_CURR_SRC value '3' ##NC

46 constants C_USER_COMPANYCODE_CURRENCY type MMPUR_REQ_D_PR_HDR_CURR_SRC value '2' ##N

47 constants C_USER_DEFAULT_CURRENCY type MMPUR_REQ_D_PR_HDR_CURR_SRC value '1' ##NO_TE

48 class-data MT_CURRENCY_SOURCE_TEXT type TY_T_CURR_SRC_TEXT .

49 class-data MO_CONFIG_PROVIDER type ref to IF_MMPUR_REQ_CONFIG_PROVIDER .

50

51 methods GET_BUFF_COMPANY_CODE_CURRENCY

52 importing

53 ITV IISERNAMF type SY-IINAMF

54

Joule

[ERI]

/explain Explain selected cl_mm_pur_common_pr_util

Joule /explain

> Used object (1)

The ABAP class CL_MM_PUR_COMMON_PR_UTIL is a utility class designed for handling common operations related to purchase requests (PR). It follows a singleton pattern with a static method GET_INSTANCE to ensure only one instance exists. The class implements the interface IF_MM_PUR_COMMON_PR_UTIL, which contains various methods for checking item authorizations, retrieving company code currency, getting the header PR currency, and fetching the PR item company code currency.

Data Declaration

The class has several data declarations including class-level data like MO_INSTANCE, MV_HDR_CURRENCY_CONFIG, MT_PR_CURRENCY_BUFFER, and MT_CURRENCY_SOURCE_TEXT. It also declares types such as ty_s_pr_currency_buffer, ty_t_pr_currency_buffer, ty_s_curr_src_text, ty_t_curr_src_text, ty_s_hierarchy_items, and ty_t_hierarchy_items.

Methods

The class consists of multiple methods:

- GET_BUFF_COMPANY_CODE_CURRENCY: Retrieves the company code currency from the buffer or fetches it using the IF_MM_PUR_COMMON_PR_UTIL~GET_COMPANY_CODE_CURRENCY method.
- GET_BUFF_ITEM_COMPANYCODE_CURR: Retrieves the item company code currency from the buffer or fetches it using the IF_MM_PUR_COMMON_PR_UTIL~GET_PR_ITEM_COMPANY_CODE_CURR method.
- GET_BUFF_USER_DEFAULT_CURRENCY: Retrieves the user default currency from the buffer or fetches it using the configuration provider.
- GET_CURRENCY_SOURCE_TEXT: Fetches the text description for the currency source.
- GET_PR_ITEM_CURRENCY: Retrieves the first item currency based on the hierarchy or directly from the PR items.
- GET_PR_ITEM_COMP_CODE_CURRENCY: Retrieves the first item company code currency based on the hierarchy or directly from the PR items.
- IF_MM_PUR_COMMON_PR_UTIL~CHECK_ITEM_AUTHORIZATION: Checks the authorization for the PR item based on the object, activity, and ID values.
- IF_MM_PUR_COMMON_PR_UTIL~GET_COMPANY_CODE_CURRENCY: Fetches the company code currency for the provided username.
- IF_MM_PUR_COMMON_PR_UTIL~GET_HEADER_PR_CURRENCY: Determines the header PR currency based on the configured source and available data.
- IF_MM_PUR_COMMON_PR_UTIL~GET_PR_ITEM_COMPANY_CODE_CURR: Fetches the company code currency for the provided PR item.

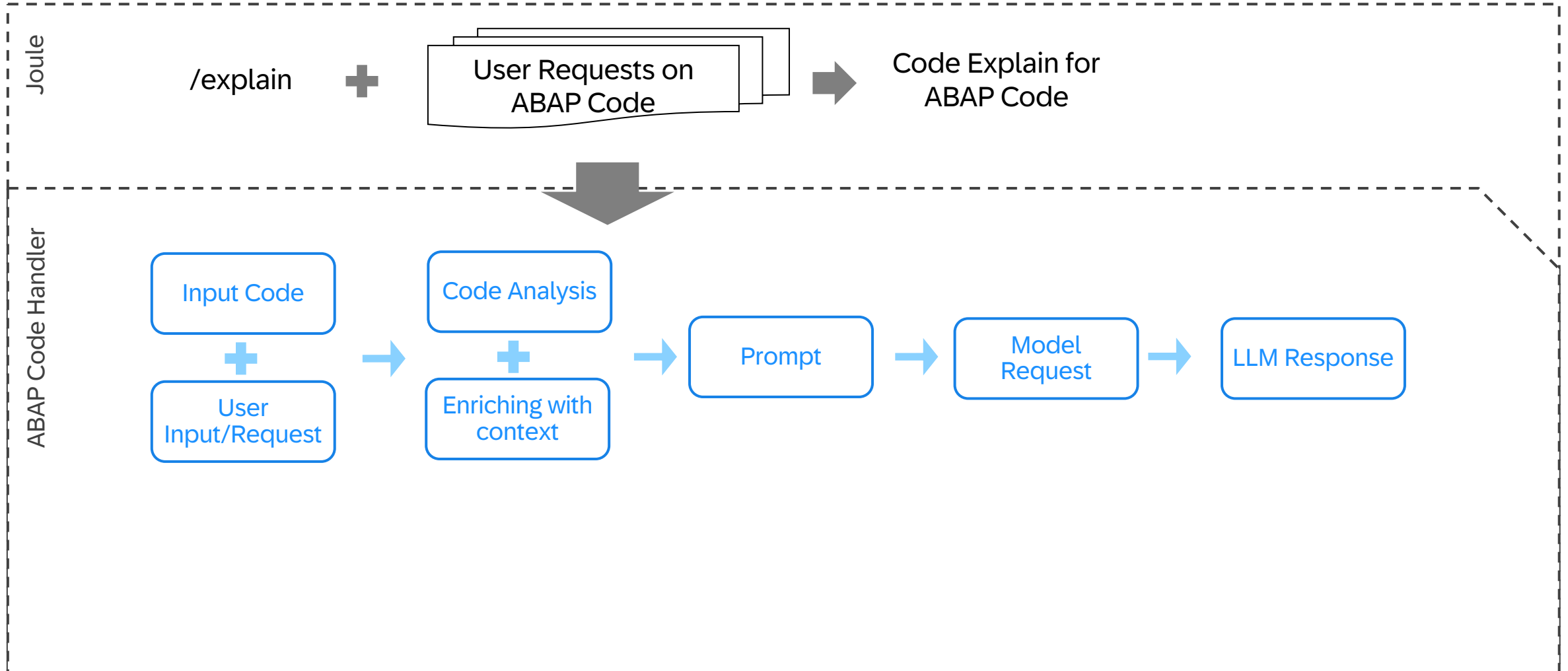
/explain

Documentation: [Get AI-Based Explanations](#) | [SAP Help Portal](#)

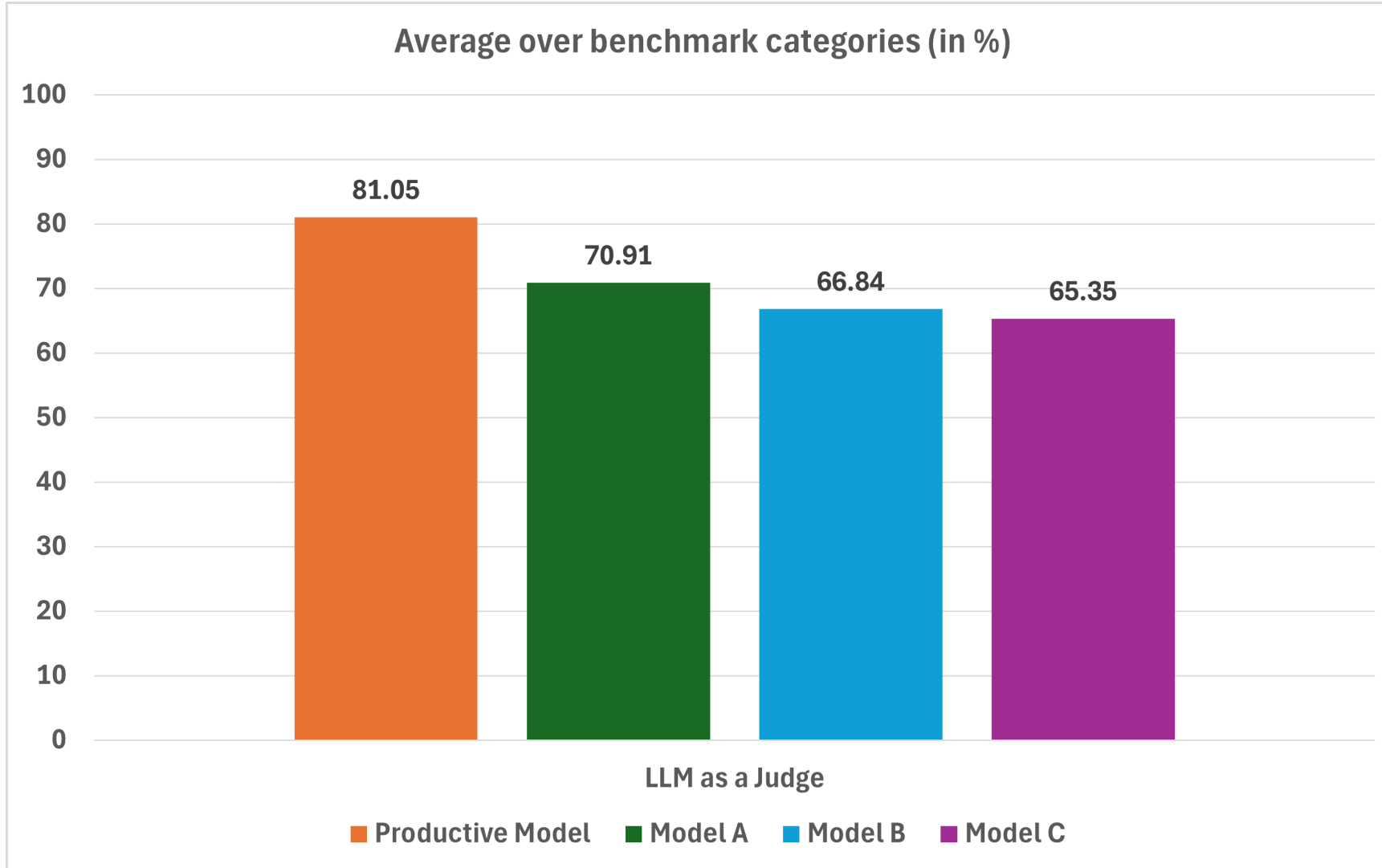
Public

4

Processing a request



Measuring model response quality



Code Explain Summary

Joule talks ABAP...

- ...it will support to understand (existing) ABAP code very quickly
- ...it will answer your specific questions on ABAP

Try it out!

The screenshot shows a chat window titled "Joule". At the top, there is a disclaimer: "Though I strive for perfection, I might not get everything right all the time. Make sure to review any generated code or suggestion." followed by a note: "This chat will assist you in your ABAP development and support tasks. Please avoid entering any personal related data, and particularly any sensitive/critical personal data (e.g. health data)." and links for "Onboarding" and "Disclaimer".

The chat history shows a user message: "/explain Can you briefly explain the business context that this class is used for and refer to each method that is implemented?".

The response from Joule is titled "Joule /explain" and includes a section "Used object (1)". The response text states: "The class CL_MM_PUR_COMMON_PR_UTIL is a utility class designed for common operations related to Purchase Requisitions (PRs) in the application. It provides various methods to handle tasks such as retrieving and setting the currency for PR items based on different configurations. Here's a brief explanation of each method:".

The response lists six methods:

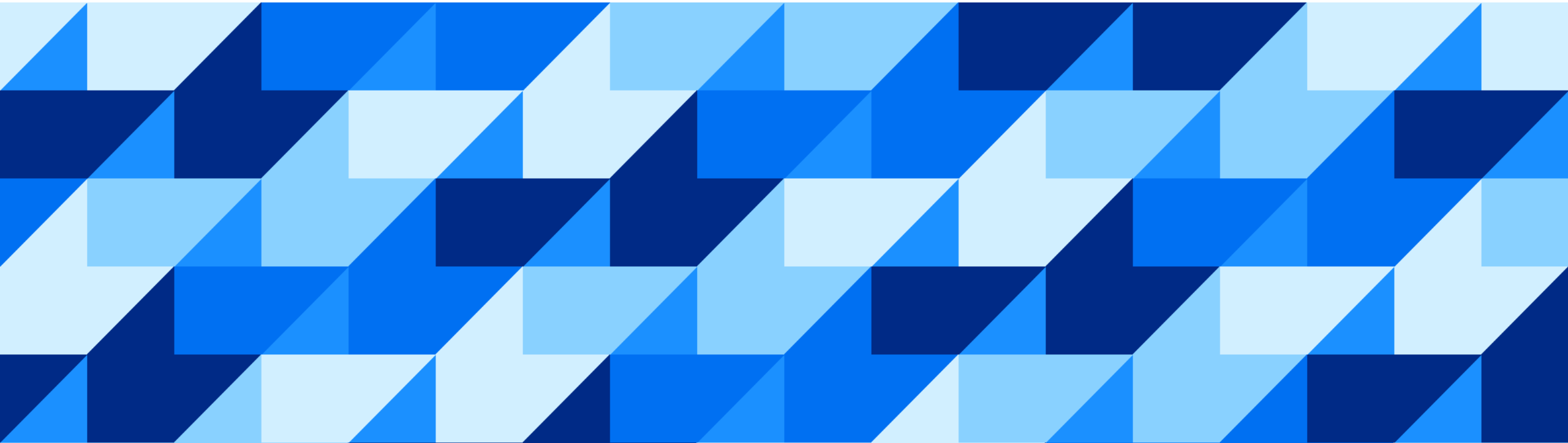
1. GET_BUFF_COMPANY_CODE_CURRENCY: This method returns the buffered company code currency for a given username.
2. GET_BUFF_ITEM_COMPANYCODE_CURR: This method returns the buffered company code currency for a specific PR item.
3. GET_BUFF_USER_DEFAULT_CURRENCY: This method returns the user's default currency by checking the buffer.
4. GET_CURRENCY_SOURCE_TEXT: This method returns the text description for a given currency source configuration.
5. GET_PR_ITEM_CURRENCY: This method retrieves the currency for the first item in a list of PR items.
6. GET_PR_ITEM_COMP_CODE_CURRENCY: This method retrieves the company code currency for the first item in a list of PR items.

At the bottom, there is an input field with the placeholder text "Ask a question." and a send button icon.

Predictive Code Completion

Manuel Berning, SAP

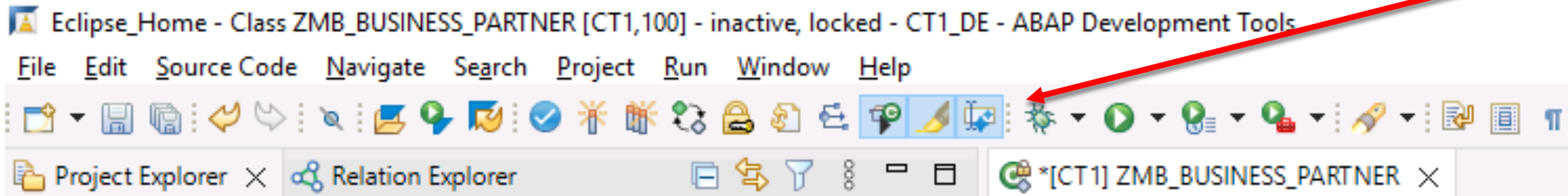
June 05, 2025



Predictive Code Completion

Try it out now!

Alternative: **Alt + J** to get a suggestion at a certain position

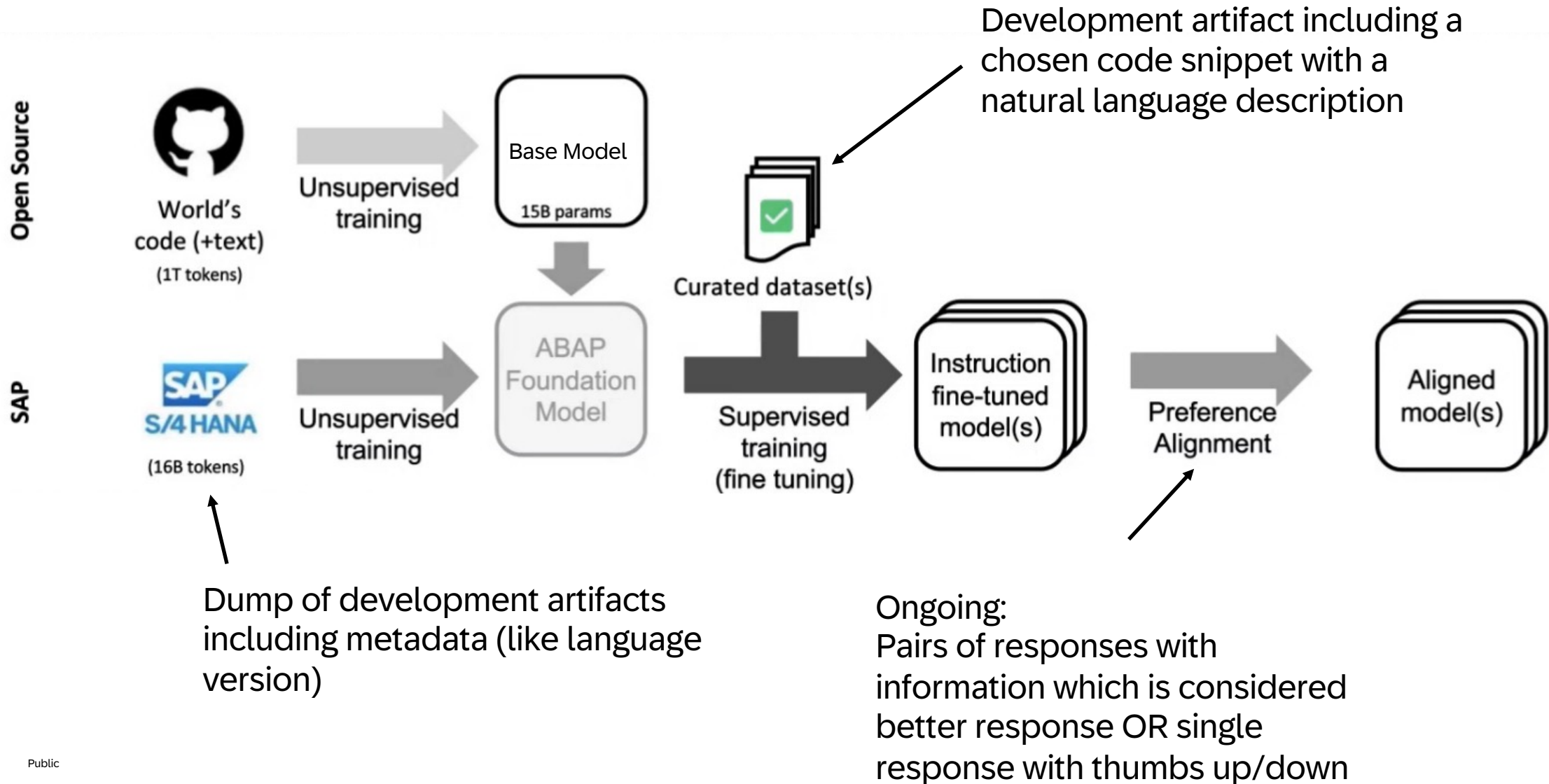


Button will only be shown when cursor is positioned in ABAP class, program or interface and backend system has been enabled!

```
METHOD run_model.  
    " Create instance of ABAP AISDK  
    DATA(api) = cl_aic_adt_completion_api_fact=>get( )->create_instance( islm_scenario = islm_scenario ).  
    " Sent a request to the model  
    DATA(result) = api->execute_for_string( prompt = prompt ).  
    answer = result->get_completion( ).  
ENDMETHOD.
```

Documentation: [Use Predictive Code Completion | SAP Help Portal](#)

Collecting relevant data for model training



Measuring model response quality

Task

- Metric:
Syntax check success rate
- 0 or 1 per task,
averaged over tasks
- Scores for this task:
Model A = 1
Model B = 0

```
CLASS s4_v3_human_abap_eval_012 DEFINITION
PUBLIC
FINAL
CREATE PUBLIC .

PUBLIC SECTION.
CLASS-METHODS:
longest
IMPORTING
strings      TYPE string_table
RETURNING
VALUE(result) TYPE string.
PROTECTED SECTION.
PRIVATE SECTION.

ENDCLASS.
```

```
CLASS s4_v3_human_abap_eval_012 IMPLEMENTATION.
```

```
METHOD longest.
```

```
* Out of list of strings, return the longest one. Return the first one in case of multiple
* strings of the same length. Return None in case the input list is empty.
* Examples
* Input: longest( VALUE string_table( ) )
* Result: VALUE string_table( )
* Input: longest(['a', 'b', 'c'])
* Result: 'a'
* Input: longest(['a', 'bb', 'ccc'])
* Result: 'ccc'
DATA(max_length) = 0.
```

Model A

```
LOOP AT strings ASSIGNING FIELD-SYMBOL(<string>).
  IF strlen( <string> ) > max_length.
    result = <string>.
    max_length = strlen( <string> ).
  ENDIF.
ENDLOOP.
```

Model B

```
LOOP AT strings INTO DATA(lv_string) INDEX lv_index.
  IF strlen( lv_string ) > max_length.
    result = lv_string.
    max_length = strlen( lv_string ).
  ENDIF.
ENDLOOP.
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

Predictive Code Completion

Measuring model response quality

Task

- Metric:
Unit test success rate
- [0 1] bounded per task, averaged over tasks
- Scores for this task:
Model A= 0.66
Model B = 1

```
CLASS zcl_human_abap_eval_004 DEFINITION
PUBLIC
FINAL
CREATE PUBLIC .

PUBLIC SECTION.
TYPES: float_table TYPE STANDARD TABLE OF decfloat16 WITH EMPTY KEY.
CLASS-METHODS:
mean_absolute_deviation
IMPORTING
numbers          TYPE float_table
RETURNING
VALUE(result) TYPE f.
PROTECTED SECTION.
PRIVATE SECTION.

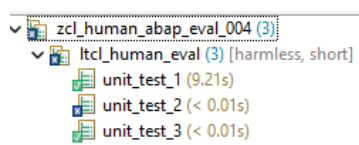
ENDCLASS.

CLASS zcl_human_abap_eval_004 IMPLEMENTATION.

METHOD mean_absolute_deviation.
* For a given list of input numbers, calculate Mean Absolute Deviation
* around the mean of this dataset.
* Mean Absolute Deviation is the average absolute difference between each
* element and a centerpoint (mean in this case):
* MAD = average | x - x_mean |
* Example
* Input: mean_absolute_deviation( VALUE float_table( ( 1.0 ) ( 2.0 ) ( 3.0 ) ( 4.0 ) ) )
* Result: 1.0
```

Model A

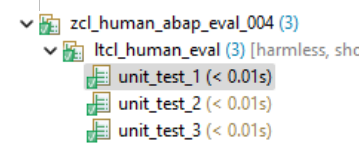
```
DATA: mean TYPE f.
mean = REDUCE f( INIT x = 0 FOR <n> IN numbers NEXT x = x + <n> ).
mean = mean / lines( numbers ).
result = REDUCE f( INIT x = 0 FOR <n> IN numbers NEXT x = x + abs( <n> - mean ) )
result = result / lines( numbers ).
```



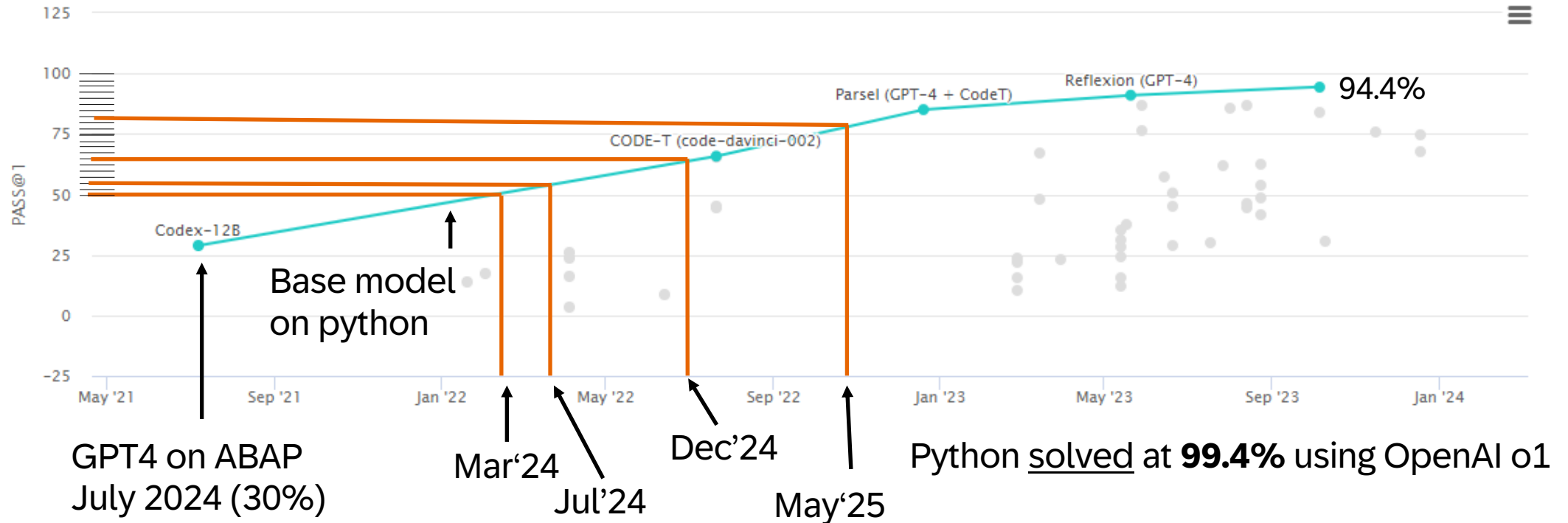
```
ENDMETHOD.
ENDCLASS.
```

Model B

```
DATA: sum TYPE f,
mean TYPE f,
deviation TYPE f,
deviations TYPE STANDARD TABLE OF f WITH EMPTY KEY.
sum = 0.
LOOP AT numbers INTO DATA(number).
sum = sum + number.
ENDLOOP.
mean = sum / lines( numbers ).
LOOP AT numbers INTO number.
deviation = abs( number - mean ).
APPEND deviation TO deviations.
ENDLOOP.
sum = 0.
LOOP AT deviations INTO deviation.
sum = sum + deviation.
ENDLOOP.
result = sum / lines( deviations ).
```



Measuring model response quality

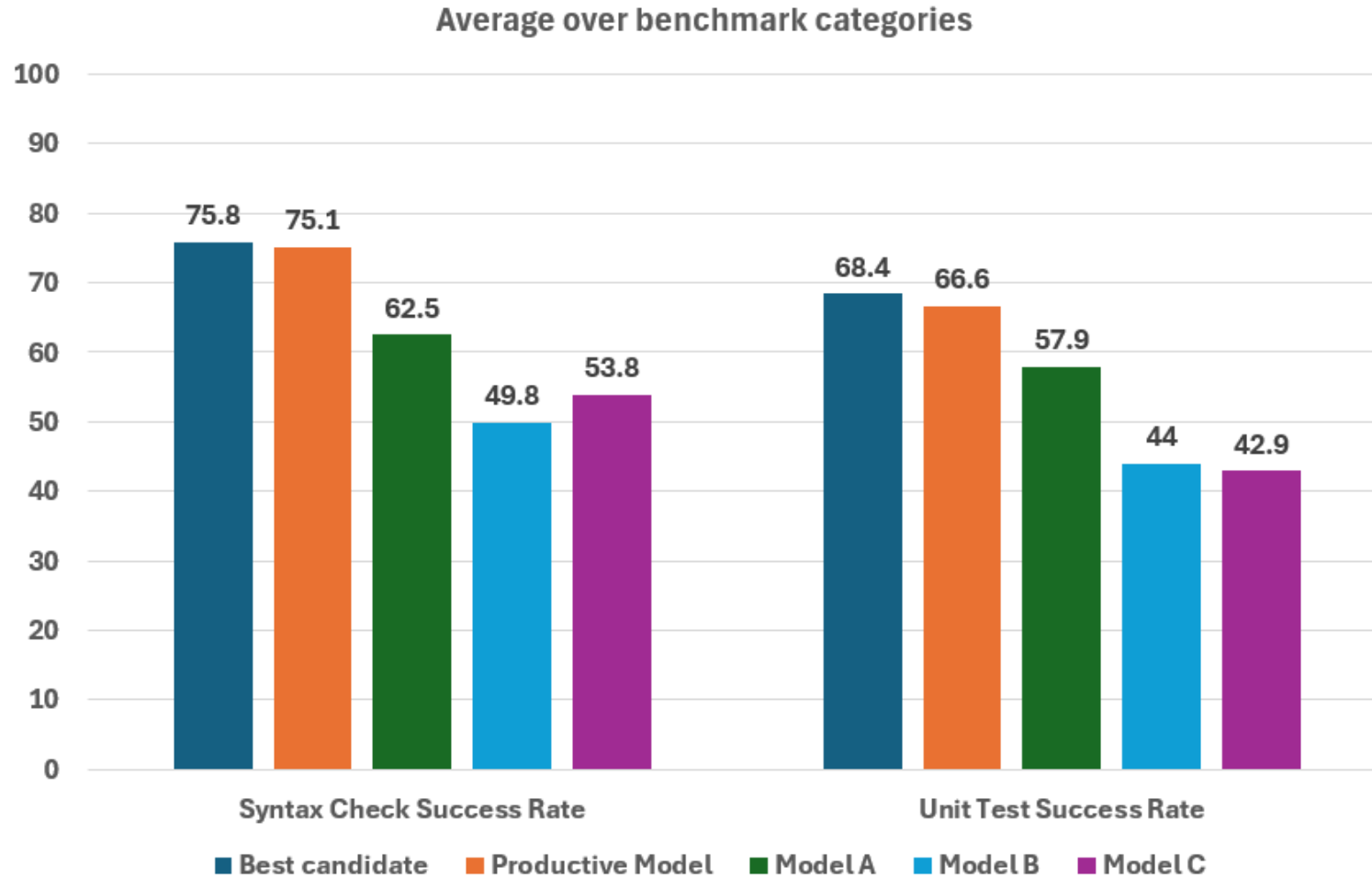


Human Eval (python) results to date:

Question: How well can an LLM implement these simple algorithms in **ABAP** in comparison?

Answer: About as good as LLM could implement these algorithms in **python** two years ago.

Measuring model response quality



- Tasks from 5 different categories
 - 398 Natural Language Comment Instructions
 - 258 S/4 Private Cloud
 - 101 Human Eval
 - 76 Public Cloud
 - 36 ABAP v5
 - 36 RAP

Predictive Code Completion

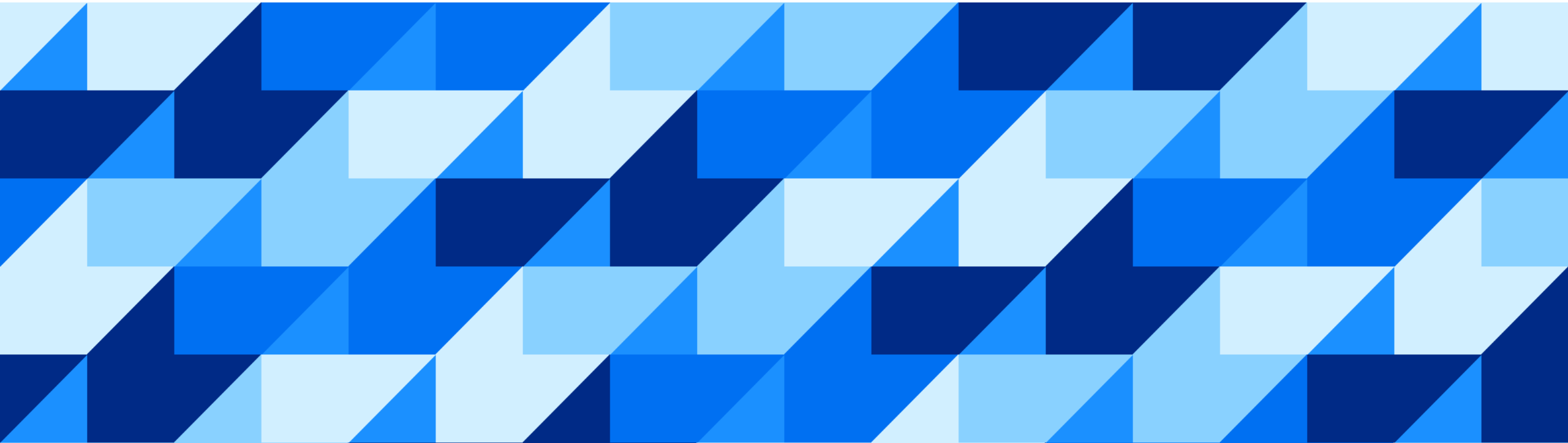
Summary

- Try it out now
 - Free SKU for customers still available for short time
- Data is key in the age of AI
 - Training on SAP ABAP codebase improves code suggestions
- Quality assurance still paramount in the age of AI
 - But different: e.g. deterministic tests vs. benchmarks
 - Track your metrics in all projects using AI

Context Injection

Leon Knorr, SAP

June 05, 2025

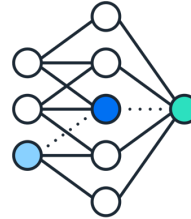


Context Injection Challenges



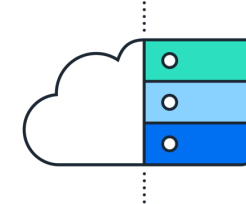
Development Environment

- Development Environments are large and complex
- Many different sources are used by developers
- Development Objects often have many dependencies



Model Limitations

- Text-Input length of models is limited due to Memory Constraints
- Long input texts might lead to information loss if not handled properly
- Long texts increase latency



Technical Challenges

- ABAP Application Server used during inference
- Python used on training infrastructure
- Challenge: How to make the Data Pipeline available for both without implementing it twice

Identifying Suitable Context

INFERENCE:

- Only Single “Dirty” Include, currently edited in ADT

- Compilation Unit (CP) as seen by the Compiler
- Interfaces for the current edited Method

- Relevant information dynamically inferred
- Removal of less relevant information based on budget, e.g. token limits
- One solution for both inference and Training

Training:

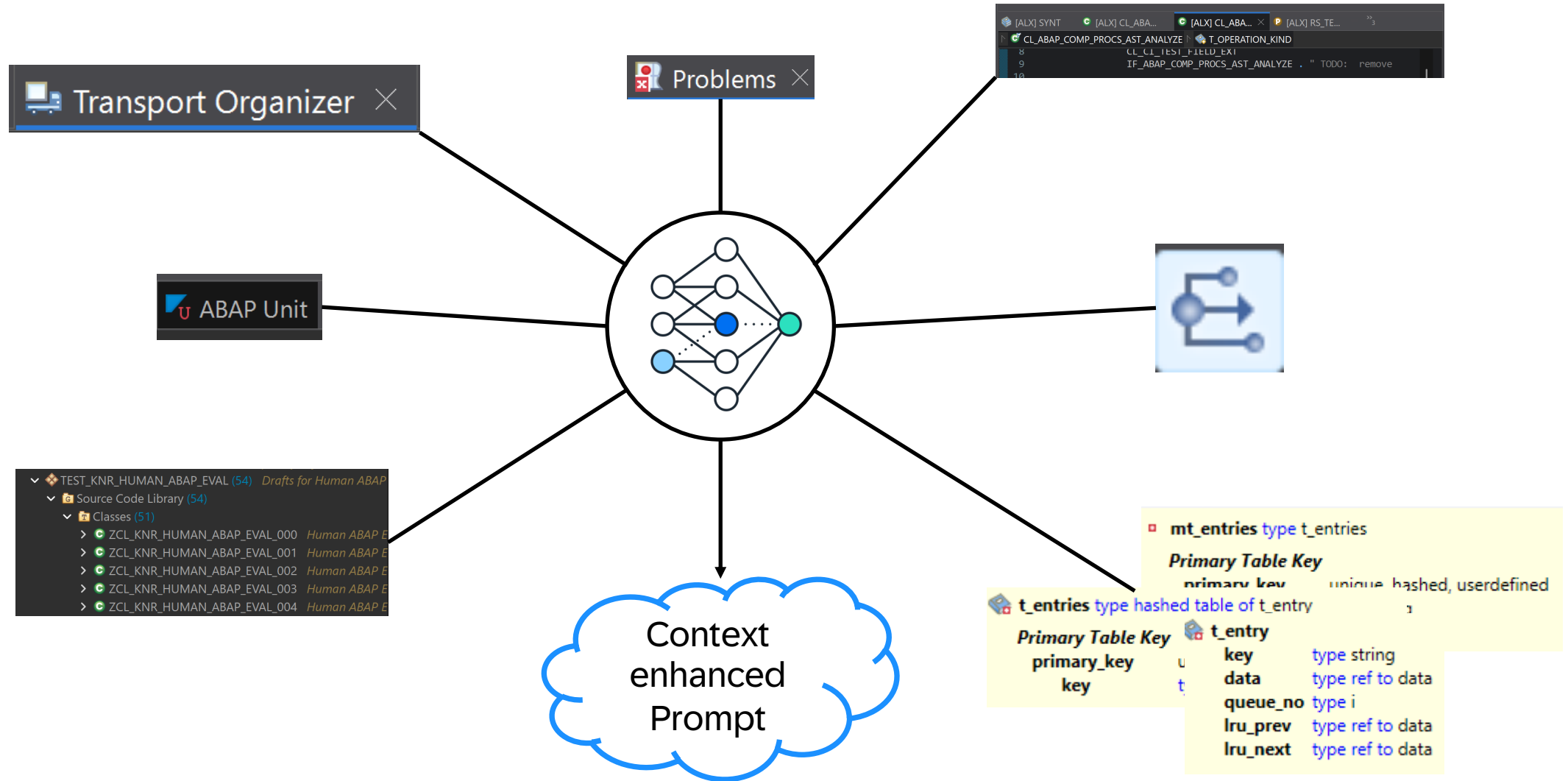
- on single include only

- on single include only
- POC for full CP ongoing

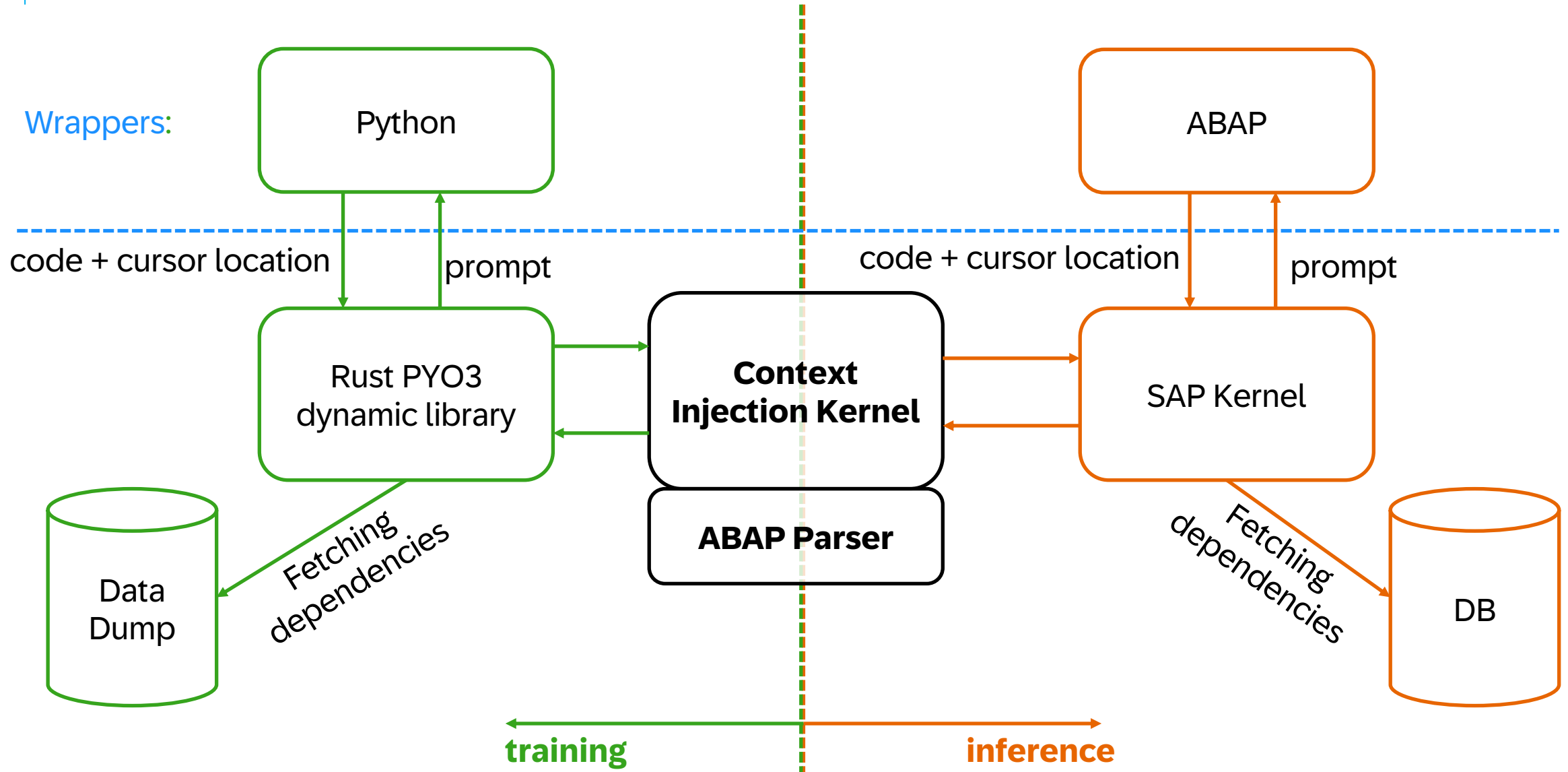


Context Injection

Identifying Suitable Context



Context Injection Architecture



Predictive Code Completion Summary

- Several technical challenges
 - Complex development environments
 - Model limitations
 - Distinct and independent environments for training and inference
- Identifying suitable context is in active development
- Fast development iterations made possible through a unified architecture in the SAP Kernel

Thank you. Questions?

Contact information:

Sevdiye Linnhoff, Leon Knorr, Manuel Berning

sevdiye.linnhoff@sap.com

leon.knorr@sap.com

manuel.berning@sap.com